Lesson:

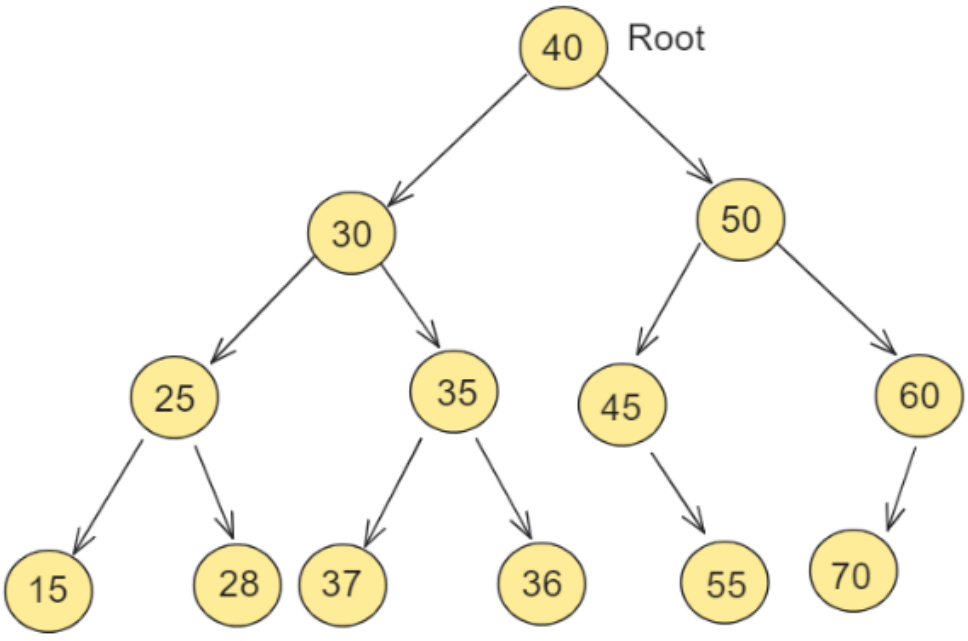
Binary Tree

Topics

- Introduction
- Implementation

Introduction

A binary tree means that the node can have a maximum of two children. Here, the binary name itself suggests that 'two'; therefore, each node can have either 0, 1, or 2 children.



Operation On Binary Tree:

Insertion:

- Compare values and traverse left or right until an empty spot is found.
- Insert the new node.

Deletion:

Find the node.

- Handle cases: no children, one child, or two children.

Searching:

- Compare values and traverse left or right until the target is found or null is reached.

Traversal:

- In-order, pre-order, and post-order to visit nodes in specific orders.

Finding Min/Max:

- Min: Traverse left subtree until no more left children.
- Max: Traverse right subtree until no more right children.

Finding Height:

- Recursive calculation of the length of the longest root-to-leaf path.

Binary Tree Traversals:

Note: we will discuss tree Traversal (preorder, inorder, postorder) in detail in the next section.

Tree Traversal algorithms can be classified broadly into two categories:

- Depth-First Search (DFS) Algorithms
- Breadth-First Search (BFS) Algorithms

Tree Traversal using Depth-First Search (DFS) algorithm can be further classified into three categories:

- **Preorder Traversal (current-left-right):** Visit the current node before exploring nodes within the left or right subtrees. The traversal sequence is root – left child – right child.
- **Inorder Traversal (left-current-right):** Visit the current node after exploring all nodes in the left subtree but before the right subtree. The traversal sequence is left child – root – right child.
- **Postorder Traversal (left-right-current):** Visit the current node after exploring all nodes in the left and right subtrees. The traversal sequence is left child – right child – root.

Tree traversal using the Breadth-First Search (BFS) algorithm is classified into one category:

- Level Order Traversal: Visit nodes level by level in a left-to-right fashion. The traversal is performed level-wise, starting from the leftmost child and moving to the right within the same level.

Implementation of Binary tree

Node Structure:
Defines a Node structure with integer data and pointers to left and right child nodes.

Tree Printing Function:
Defines a function Printtree to print the binary tree using an inorder traversal.

Main Function:
Creates a binary tree with root value 1, left child 2, and right child 3.
Inserts node 4 as the left child of node 2.

Printing the Tree:
Calls the Printtree function to print the binary tree.

JavaScript

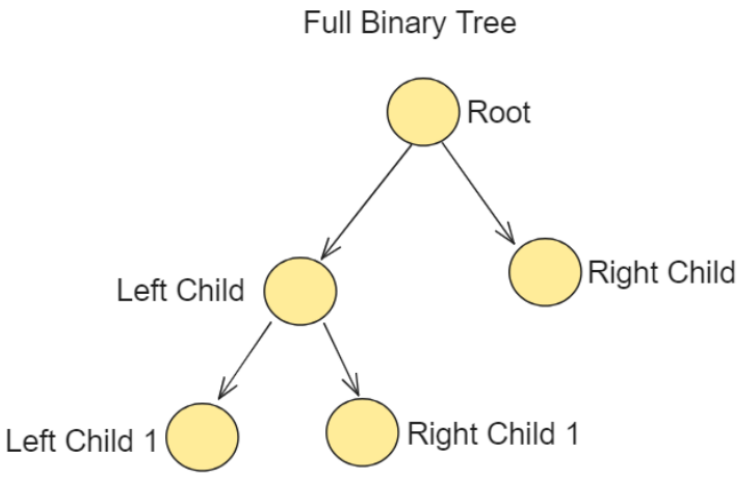
```
// A JavaScript program to implement a binary tree in data
structures

// A class to create node
class Node {
  constructor(value) {
    this.data = value;
    this.left = null; // Left child is initialized to null
    this.right = null; // Right child is initialized to null
  }
}
```

```
// A function to print the tree
function printTree(root) {
  // Check if the tree is empty
  if (root === null) return;

  // Use inorder traversal to print the tree
  printTree(root.left);
  console.log(root.data + " ");
  printTree(root.right);
}

// Creating root
const root = new Node(1);

/*
   (1)
  / \
null null
*/

       (1)
      / \
   (2)   (3)
  / \    / \
null null null null
*/

root.left.left = new Node(4); // Inserting 4 as the left child of
2

/*
       (1)

       / \
    (2)   (3)
   / \
 (4) null
 */

// Function call to print the tree
printTree(root);  // 4 2 1 3
```

Types of binary search tree

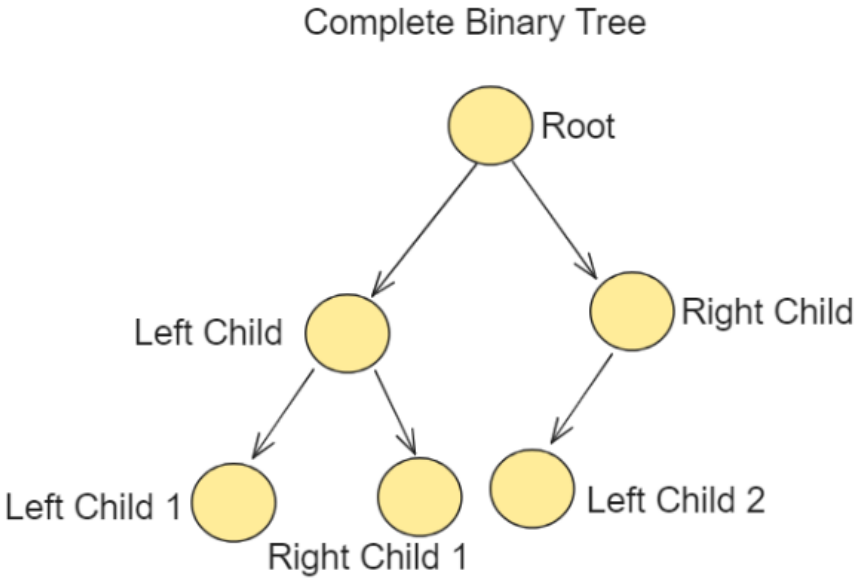
Full Binary Tree

A full binary tree (sometimes called a proper or plane binary tree) is a tree in which every node has either 0 or 2 children. In other words, every node in a full binary tree has exactly 0 or 2 children, and all leaf nodes are at the same level.



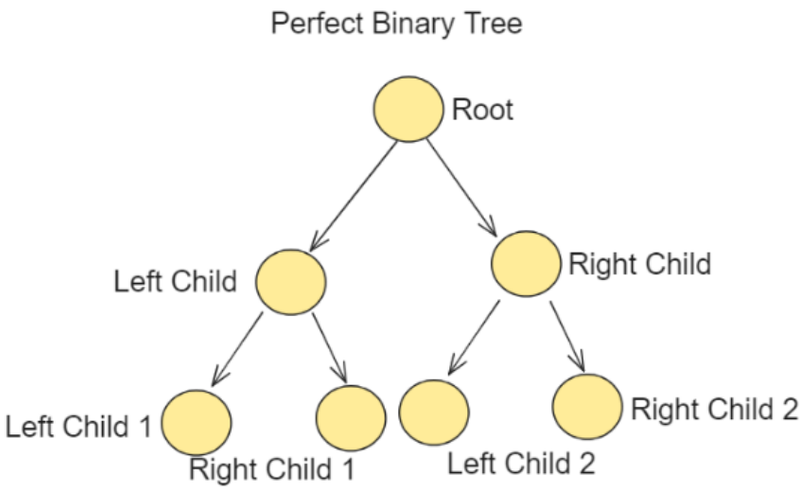
Complete Binary Tree

A complete binary tree is a binary tree in which every level, except possibly the last, is completely filled, and all nodes are as left as possible. In other words, it is a binary tree in which the nodes are filled from left to right on each level.



Perfect Binary Tree

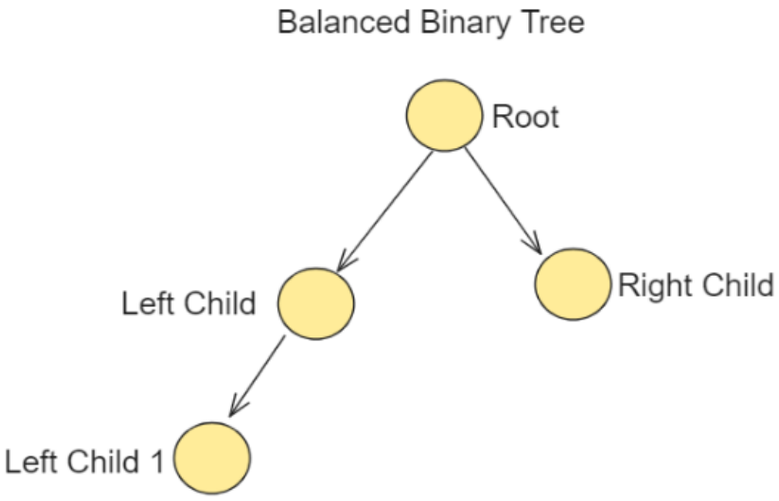
A perfect binary tree is a binary tree in which all the levels are completely filled, and each level has the maximum number of nodes possible. In other words, a perfect binary tree is a complete binary tree where all levels are completely filled, and there are no missing nodes.



Balanced Binary Tree

A balanced binary tree is a binary tree in which the depth of the two subtrees of every node never differs by more than one. In other words, a balanced binary tree is a binary tree where the height of the left and right subtrees of any node differ by at most one.

Balance factor = height(left subtree) - height(right subtree)



Degenerate Binary Tree

A degenerate (or pathological) binary tree is a binary tree in which each parent node has only one associated child node. Essentially, it is a tree that is skewed to one side, either left or right, and resembles a linked list rather than a balanced tree.